

Reinforcement Learning Reviewer

Modules 9, 10, 11 & Activity Walkthroughs

Made by James Adrian B. Castro

A comprehensive reviewer covering:

- Dynamic Programming & Planning
- Monte Carlo Tree Search (MCTS)
- Q-Learning & Temporal Difference
- Activity Walkthroughs & Practice Problems

MODULE 9

Dynamic Programming, Planning & Reinforcement

1. Markov Decision Processes (MDPs)

MDPs formally describe an environment for reinforcement learning where the environment is fully observable.

Almost all RL problems can be formalized as MDPs.

MDPs are considered 'solved' if an optimal policy is obtained.

The Markov Property

"The future is independent of the past given the present."

$$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, \dots, S_t]$$

- Once the state is known, history may be thrown away
- The current state is a sufficient statistic of the future

MDP Definition

A Markov Decision Process is a tuple $\langle S, A, P, R, \gamma \rangle$:

Symbol	Meaning
S	Finite set of states
A	Finite set of actions
P	State transition probability matrix
R	Reward function
γ	Discount factor (0 to 1)

$$P_{ss'}^a = P[S_{t+1}=s' | S_t=s, A_t=a]$$

$$R_s^a = E[R_{t+1} | S_t=s, A_t=a]$$

- Each row of the state transition matrix sums to 1

2. Dynamic Programming (DP)

DP is a method for solving complex problems by breaking them down into subproblems.

- Solve the subproblems
- Combine solutions to subproblems

DP = collection of algorithms that compute optimal policies given a perfect model of the environment as an MDP.

Two Properties of DP

- Optimal substructure — optimal solution can be decomposed into subproblems
- Overlapping subproblems — subproblems recur many times; solutions can be cached and reused

MDPs satisfy both properties: Bellman equation gives recursive decomposition, and the value function stores

and reuses solutions.

3. The Bellman Equation

$$V(s) = \max_a [R(s,a) + \gamma V(s')]$$

- Breaks a large problem into smaller state-value problems (Optimal Substructure)
- Stores computed state values for reuse (Overlapping Subproblems)

4. Planning

Planning is any computational process that uses a model to create or improve a policy.

Model → Simulated Experience → Backups → Values → Policy

Direct vs Indirect RL

Type	Purpose
Direct RL	Directly improve value function and policy
Indirect RL	Improve the learned model (model-based RL / planning)

5. Planning by DP: Prediction & Control

Component	Description
Prediction	Learn value function for the policy followed
Control	Find a policy that maximizes reward

6. Iterative Policy Evaluation

Problem: evaluate a given policy. Solution: iterative application of Bellman expectation formula.

$$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_\pi$$

7. Policy Improvement & Policy Iteration

Given policy π : evaluate the policy, then improve by acting greedily.

$$\pi_0 \rightarrow v_{\{\pi_0\}} \rightarrow \pi_1 \rightarrow v_{\{\pi_1\}} \rightarrow \pi_2 \rightarrow \dots \rightarrow \pi^* \rightarrow v^*$$

Each policy is guaranteed to be a strict improvement over the previous one (unless already optimal).

MODULE 10

Monte Carlo Tree Search (MCTS)

1. Introduction

MCTS is a best-first search method that does not require a positional evaluation function. It is based on a randomized exploration of the search space.

- Uses results of previous explorations to gradually build a game tree in memory
- Successively becomes better at estimating values of the most promising moves
- Heavily relies on randomness ('Monte Carlo' from the gambling district in Monaco)

Planning time is independent of size of state space |S|.

2. Tic-Tac-Toe Example

Possible moves: A, B, C

Move	Wins	Losses	Draws	Win Rate
A	40	35	25	40%
B	60	20	20	60%
C	90	5	5	90%

3. The Four Phases of MCTS

Four phases iterated for a determined number of times: Selection → Expansion → Simulation → Backpropagation.

Phase 1: Selection

- Starting at root, select a node based on the biggest UCB value
- UCB formula balances exploitation and exploration:

$$UCB = w_i / s_i + c * \sqrt{\ln(s_p) / s_i}$$

Phase 2: Expansion

- If a node has been visited, its children are generated and added to the tree

Phase 3: Simulation

- If selected node is expanded, take a random child; else take the selected node itself
- Randomly choose next states until reaching a terminal state

Phase 4: Backpropagation

- The value from simulation is propagated from leaves to root
- Updates the values of the nodes visited

4. Why Randomness Works

Law of Large Numbers (LLN): From a single random game, very little can be learned. But from simulating a multitude of random games, a good strategy can be inferred.

MODULE 11

Q-Learning & Temporal Difference

1. Monte Carlo Limitation

Monte Carlo RL overcomes unknown model difficulty, but a disadvantage is that the strategy can only be updated after the whole episode.

2. Temporal Difference (TD) Learning

TD Learning is a combination of Monte Carlo ideas and Dynamic Programming ideas.

- Like MC: can learn directly from raw experience without a model
- Like DP: updates estimates based in part on other learned estimates, without waiting for a final outcome

TD Update Formula

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD Error: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

3. Behavior Policy vs Target Policy

Policy	Definition
Behavior Policy	Policy the agent uses for action selection
Target Policy	Policy the agent is trying to learn

4. On-Policy vs Off-Policy Learning

On-Policy	Off-Policy
'Learn on the job'	'Look over someone's shoulder'
Target policy == Behavior policy	Target policy != Behavior policy
Experiences collected using latest policy	Can use older samples

5. Q-Learning

Q stands for Quality — how useful a given action is in gaining future reward.

Q-Learning Update

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Q-Learning Algorithm

```
Initialize Q(s,a) arbitrarily
Repeat (for each episode):
  Initialize s
```

```

Repeat (for each step of episode):
  Choose a from s using policy derived from Q
  Take action a, observe r, s'
   $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ 
   $s \leftarrow s'$ 
until s is terminal

```

6. Deep Q-Learning (DQN)

When the state space is too large for a Q-table, use a neural network to approximate Q-values.

Q-Learning	Deep Q-Learning
Q-table	Neural network
State + Action $\rightarrow$ Q-value	State $\rightarrow$ Q-values for all actions
Tabular	Function approximation

MODULE A

Appendix: Practice Problems & Activity Walkthroughs

1. Direct vs Indirect RL & Prediction Problems

Direct vs Indirect RL Answers

- Method A uses Direct RL.
- Method B uses Indirect RL.
- Advantage of Method B (Indirect): Can plan routes using the map.
- Advantage of Method A (Direct): Simpler; avoids errors of an imperfect model.

Choosing the Best Approach:

- Indirect RL is more suitable (model-based). It simulates extra experiences in a model.
- It performs better because it can improve performance over time without many real game plays.
- Risk: The model will be inaccurate, leading to poor planning and poor suboptimal decisions.

Prediction Problem Answers

- State with highest future reward: State A (value = 20).
- Least desirable state: State C (value = 5).
- Value functions tell us the cumulative future reward from each state under the current policy. A higher value means the State is desirable.
- Optimal Route: Route B (Expected Reward 80 vs Route A's 10).
- Planning Component: Control (involves choosing the best route possible to maximize a reward, not just evaluating a fixed policy).

2. Bellman Equations & Value Iteration

Bellman Equations for a Markov Reward Process

$$V(s) = R(s) + \gamma \sum_{s'} P(s'|s) V(s')$$

Specific State Equations:

$$\begin{aligned} V(\text{YouTube}) &= -3 + 0.9 \sum P(\text{YouTube} \rightarrow s') V(s') \\ V(\text{Study1}) &= -1 + 0.9 \sum P(\text{Study1} \rightarrow s') V(s') \\ V(\text{Study2}) &= -1 + 0.9 \sum P(\text{Study2} \rightarrow s') V(s') \\ V(\text{Cafeteria}) &= 2 + 0.9 \sum P(\text{Cafeteria} \rightarrow s') V(s') \\ V(\text{Project}) &= 4 + 0.9[0.7V(\text{Graduate}) + 0.3V(\text{Cafeteria})] \\ V(\text{Graduate}) &= 15 + 0.9(1)V(\text{Sleep}) \\ V(\text{Sleep}) &= 0 + 0.9 \sum P(\text{Sleep} \rightarrow s') V(s') \end{aligned}$$

Calculation of Graduate and Sleep Values

Given that Graduate always goes to Sleep, and if Sleep is terminal/absorbing ($V(\text{Sleep}) = 0$):

$$V(\text{Graduate}) = 15 + 0.9(0) = 15$$

Answer: $V(\text{Graduate}) = 15$, $V(\text{Sleep}) = 0$

Value Iteration (1st Iteration with $V_0(s) = 0$)

Bellman update becomes $V_1(s) = R(s)$.

State	$V_1(s)$
YouTube	-3
Study1	-1
Study2	-1
Cafeteria	+2
Project	+4
Graduate	+15
Sleep	0

- Value of Project after the first iteration: $V_1(\text{Project}) = 4$
- State with the highest long-term value after this iteration: Graduate (15)

3. TD(0), Monte Carlo, and Q-Learning Calculations

Problem 1: Temporal Difference TD(0)

Given: $V(A)=4$, $V(B)=7$, $R=3$, $\alpha=0.5$, $\gamma=0.9$

$$V(A) \leftarrow 4 + 0.5 [3 + 0.9(7) - 4]$$

$$= 4 + 0.5 [3 + 6.3 - 4]$$

$$= 4 + 0.5 [5.3]$$

$$= 4 + 2.65 = 6.65$$

Result: State A = 6.65

Problem 2: Monte Carlo Learning

Given: $V(A)=5$, Total return $R_t = 10$, $\alpha = 0.4$

$$V(A) \leftarrow 5 + 0.4 [10 - 5]$$

$$= 5 + 0.4(5)$$

$$= 5 + 2 = 7$$

Result: New Value of location A = 7

Problem 3: Q-Learning

Given: Action Left=3, Right=8, Forward=6. $Q(A, \text{Right})=4$, $R=5$. Next State = B. Q-values in Room B: $\max(3, 8, 6) = 8$.

$$Q(A, \text{Right}) \leftarrow 4 + 0.5 [5 + 0.9(8) - 4]$$

$$= 4 + 0.5 [5 + 7.2 - 4]$$

$$= 4 + 0.5 [8.2]$$

$$= 4 + 4.1 = 8.1$$

Result: $Q(A, \text{Right}) = 8.1$

4. MCTS Application: DFS Process

Problem: Consider the following tree. Use the DFS process (Selection → Expansion → Simulation → Backup) to find a path from the root node A to the goal node K.

Tree Structure:

Root A → Left child B, Right child C
B → Left child D (leaf/failure), Right child E
E → Left child J (leaf/failure), Right child K (goal node)

Step 1: Selection

- Start at the root node A.
- Choose the leftmost unexpanded child.
- Path selected: $A \rightarrow B \rightarrow D$ (D is a leaf/failure).
- Next unexpanded child: $A \rightarrow B \rightarrow E$.

Step 2: Expansion

- Expand Node E to get children J and K.

Step 3: Simulation

- $A \rightarrow B \rightarrow E \rightarrow J$ (leftmost child = failure since J is not our goal).
- $A \rightarrow B \rightarrow E \rightarrow K$ (Goal node).
- Shortened Form: $E \rightarrow K$ (check if K is the goal node).

Step 4: Backup

Propagate the success up the tree.

Node	Result
K	Success
E	Success
B	Success
A	Success